

OFFICIAL GRADING RUBRIC**Total Score: 10.0 Points (100%)****Part I (Quiz): 6.0 pts (12 × 0.5)****Part II (Code): 4.0 pts (2 problems)**

NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE & AI

MIDTERM EXAMINATION SOLUTIONS**Course:** Data Analysis with Python (DSAI1005)**Exam Code:** 301-EN — **Duration:** 60 minutes**PART I: MULTIPLE-CHOICE SOLUTIONS (6.0 pts — 0.5 pt each)****Master Multiple-Choice Answer Key**

Q#	Topic	Weight	Key	Core Concept Evaluated
Q1	NumPy	0.5 pt	A	<code>np.linspace</code> step calculation and element-wise vectorization.
Q2	NumPy	0.5 pt	B	Step slicing <code>[:, -1, 1]</code> memory view mutation versus copy behavior.
Q3	NumPy	0.5 pt	A	Multi-dimensional broadcasting across shapes (4, 1, 6) and (3, 1).
Q4	NumPy	0.5 pt	A	Conditional filtering with <code>np.where</code> and column reduction <code>axis=0</code> .
Q5	Pandas	0.5 pt	B	Label-aligned addition with <code>s1.add(s2, fill_value=0)</code> .
Q6	Pandas	0.5 pt	A	Boundary semantics: inclusive label slicing <code>.loc</code> vs half-open <code>.iloc</code> .
Q7	Pandas	0.5 pt	B	Multi-level grouping <code>groupby</code> and multi-metric aggregation <code>.agg</code> .
Q8	Pandas	0.5 pt	B	SQL-style left join semantics: preserving left row count with right NaN.
Q9	Viz	0.5 pt	A	Object-Oriented 2D subplot array indexing <code>axes[row, col]</code> and methods.
Q10	Viz	0.5 pt	B	Continuous distributions with extreme skewness: boxplot 5-number summary.
Q11	Viz	0.5 pt	A	Seaborn semantic channels: <code>hue</code> for palette colors, <code>style</code> for glyphs.
Q12	Viz	0.5 pt	B	Heatmap numeric value overlay (<code>annot=True</code>) and layout adjustments.

Step-by-Step Technical Explanations

- **Q1 (Option A):** `np.linspace(0, 10, 5)` yields array `[0.0, 2.5, 5.0, 7.5, 10.0]`. `np.arange(1, 6)` produces integer array `[1, 2, 3, 4, 5]`. Scaled by 2, $b \times 2 = [2, 4, 6, 8, 10]$. Vectorized addition produces `[0 + 2, 2.5 + 4, 5.0 + 6, 7.5 + 8, 10.0 + 10] = [2.0, 6.5, 11.0, 15.5, 20.0]`. Floating-point promotion applies.
- **Q2 (Option B):** Basic slicing on ndarrays produces a *view*, sharing underlying data buffer with `m`. The slice `m[:, -1, 1]` extracts column 1 in reversed order: row index 2 (80), row index 1 (50), and row index 0 (20). Assigning `v[0] = 999` mutates the corresponding element in `m`, which is `m[2, 1]`. Meanwhile, `v[2]` corresponds to `m[0, 1] = 20`. Thus `m[2, 1]` is 999 and `v[2]` is 20.
- **Q3 (Option A):** Broadcasting aligns trailing axes: $A : (4, 1, 6)$ and $B : (3, 1)$. Adding leading singleton: $B : (1, 3, 1)$. Comparing dimensions: axis 2 has $(6, 1) \rightarrow 6$; axis 1 has $(1, 3) \rightarrow 3$; axis 0 has $(4, 1) \rightarrow 4$. Result shape of $A \times B$ is $(4, 3, 6)$. Adding `np.zeros(6)` aligns with axis 2 (size 6), preserving shape $(4, 3, 6)$.
- **Q4 (Option A):** Condition: $(x \% 2 == 0) \wedge (x > 10)$. Evaluating matrix $x = [[12, 25, 8], [30, 15, 22]]$:
 - Row 0: 12 is even and $> 10 \rightarrow 12 \times 2 = 24$; 25 is odd $\rightarrow 0$; 8 is $\leq 10 \rightarrow 0$.
 - Row 1: 30 is even and $> 10 \rightarrow 30 \times 2 = 60$; 15 is odd $\rightarrow 0$; 22 is even and $> 10 \rightarrow 22 \times 2 = 44$.`filtered` is `[[24, 0, 0], [60, 0, 44]]`. Column sum along `axis=0`: `[24 + 60, 0 + 0, 0 + 44] = [84, 0, 44]`.

- **Q5 (Option B):** `.add(..., fill_value=0)` replaces missing values with 0 prior to addition. For 'Da Nang', S_1 has 30 and S_2 is missing $\rightarrow 30 + 0 = 30.0$. For 'Can Tho', S_1 is missing and S_2 has 15 $\rightarrow 0 + 15 = 15.0$.
- **Q6 (Option A):** `.loc` uses label-based indexing where both start and stop endpoints are inclusive ('Q2', 'Q3', 'Q4' $\rightarrow$ 3 rows). `.iloc` uses 0-based integer position indexing where the stop endpoint is exclusive (`[1, 3)` selects indices 1 and 2 $\rightarrow$ 2 rows).
- **Q7 (Option B):** Rows matching ('North', 'Food') are row 0 (Sales=100) and row 4 (Sales=200). Total sum is $100 + 200 = 300$.
- **Q8 (Option B):** A left join guarantees retention of all records from the left DataFrame (A , 500 rows). Unmatched foreign keys will populate attributes from B with NaN.
- **Q9 (Option A):** In a 2D subplot array `axes`, indexing follows matrix notation `[row, col]`. Row index 0 is row 1; column index 1 is column 2. Thus `axes[0, 1].set_title(...)` correctly invokes the OO method.
- **Q10 (Option B):** A boxplot accurately conveys non-parametric distributions via median, quartiles ($Q1, Q3$), and flags extreme values beyond $1.5 \times IQR$ as individual points, avoiding distortion from heavy tails.
- **Q11 (Option A):** Seaborn maps `hue` to distinct color palettes, and `style` to point marker glyphs (e.g., circle, square, triangle).
- **Q12 (Option B):** `annot=True` renders the numeric values directly inside each matrix cell; `plt.tight_layout()` automatically computes subplot margins to prevent axes labels and titles from overlapping or clipping.

PART II: APPLIED CODING SOLUTIONS & GRADING RUBRIC (4.0 pts)

Problem 1: Quantitative E-Commerce Performance Pipeline (2.0 pts)

Standard Model Solution

```
# [BLANK 1] (0.4 pt)
(tx_data[:, 0] * tx_data[:, 1] * (1 - tx_data[:, 2])) - tx_data[:, 3]

# [BLANK 2] (0.4 pt)
np.where((net_revenue >= 250000) | (tx_data[:, 1] >= 4), 'Wholesale', 'Retail')

# [BLANK 3] (0.4 pt)
cust_df, on='cust_id', how='left'

# [BLANK 4] (0.4 pt)
['city', 'tier']]['net_rev'].agg(['sum', 'count'])

# [BLANK 5] (0.4 pt)
('net_rev', 'sum'), ascending=False
```

Grading Criteria & Scoring Rubric for Problem 1

- **[BLANK 1] (0.4 pt total):**
 - **0.2 pt:** Accurate 2D slice indexing: `tx_data[:, 0]`, `tx_data[:, 1]`, `tx_data[:, 2]`, `tx_data[:, 3]`. (Deduct 0.1 pt if student wrote 1D index like `tx_data[0]`).
 - **0.2 pt:** Correct algebraic net revenue formula respecting parentheses and order of operations.
 - *Alternative:* `(tx_data[:, 0] * tx_data[:, 1] * (1 - tx_data[:, 2])) - tx_data[:, 3]` expanded algebraically is fully accepted (0.4 pt).
- **[BLANK 2] (0.4 pt total):**
 - **0.2 pt:** Correct conditional with bitwise OR `|` and proper parentheses around individual boolean clauses: `(net_revenue >= 250000) | (tx_data[:, 1] >= 4)`. Deduct 0.1 pt if Python keyword `or` was used (causes `ValueError` on arrays).
 - **0.2 pt:** Exact string labels `'Wholesale'` and `'Retail'` in true and false branches.
- **[BLANK 3] (0.4 pt total):**
 - **0.2 pt:** Target merge table `cust_df` and join key `on='cust_id'` specified.
 - **0.2 pt:** Join type `how='left'` specified.
- **[BLANK 4] (0.4 pt total):**
 - **0.2 pt:** Grouping keys list `['city', 'tier']` (or tuple `('city', 'tier')`).
 - **0.2 pt:** Target column selection `['net_rev']` chained with aggregation `.agg(['sum', 'count'])`.
 - *Alternative:* Dictionary syntax `merged.groupby(['city', 'tier']).agg({'net_rev': ['sum', 'count']})` is fully accepted for full credit.
- **[BLANK 5] (0.4 pt total):**
 - **0.2 pt:** MultiIndex column tuple specification `('net_rev', 'sum')` or `[('net_rev', 'sum')]`.
 - **0.2 pt:** Sorting direction parameter `ascending=False`.

Problem 2: Multi-Channel Sales Dashboard & Risk Analytics (2.0 pts)**Standard Model Solution**

```
# [BLANK 6] (0.4 pt)
plt.subplots(1, 2, figsize=(14, 5))

# [BLANK 7] (0.6 pt)
data=df_sales, x='payment', y='amount', hue='is_fraud', palette='Set2', ax=axes[0]

# [BLANK 8] (0.4 pt)
rolling(window=4).mean() # or: rolling(4).mean()

# [BLANK 9] (0.2 pt)
plt.tight_layout()
```

Part C: Executive Insights Model Answers (0.4 pt)**Question 1: Outlier Identification in Boxplot (0.2 pt)**

- **Model Answer:** In a boxplot, typical legitimate transactions fall inside the central box (the Interquartile Range from 25th percentile Q_1 to 75th percentile Q_3) and within the upper and lower whiskers (defined as $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$). Any transactions extending beyond the whisker thresholds are marked as individual outlier points (fliers), visually highlighting anomalous or suspicious transaction values that warrant fraud investigation.
- **Grading:** Award 0.1 pt for mentioning IQR / box boundary; award 0.1 pt for noting that points plotted past $1.5 \times IQR$ / whiskers represent outliers.

Question 2: Forecasting Advantage of Rolling Average (0.2 pt)

- **Model Answer:** Raw weekly sales data contains high-frequency random noise, day-of-week timing quirks, and one-off anomalies that create erratic peaks and valleys. A 4-week rolling moving average dampens these short-term fluctuations by smoothing the trend, revealing the true underlying baseline trajectory and momentum essential for robust quarterly revenue projections.
- **Grading:** Award 0.1 pt for mentioning smoothing/reducing variance or noise; award 0.1 pt for explaining that it reveals the underlying trend for forecasting.

— END OF SOLUTION & GRADING GUIDE —